```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

print("TensorFlow version:", tf.__version__)
print("Libraries imported successfully!")
```

```
TensorFlow version: 2.21.0
Libraries imported successfully!
```

```
# Load MNIST dataset
(x_train_full, y_train_full), (x_test, y_test) = (
    tf.keras.datasets.mnist.load_data()
)

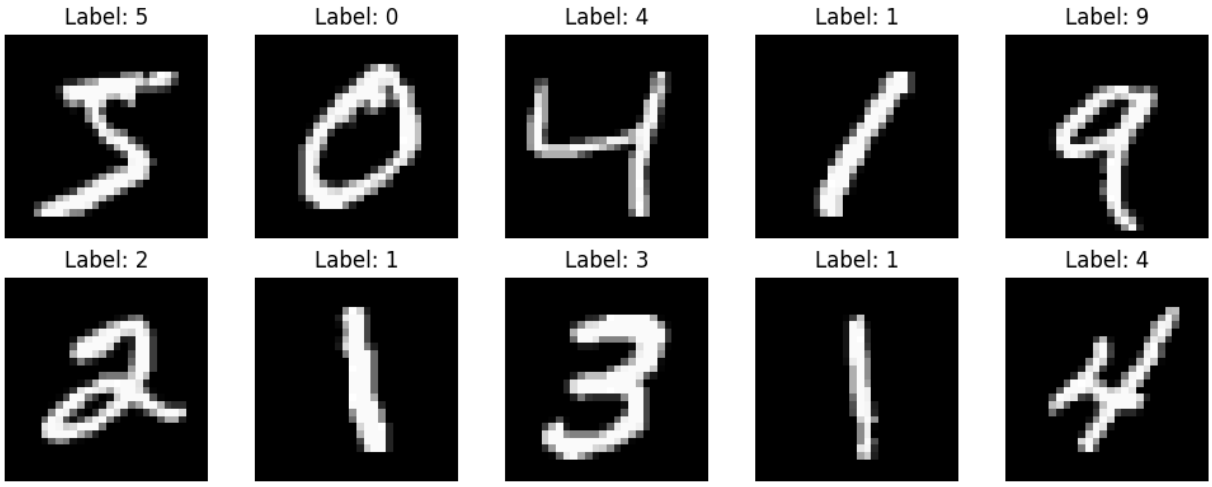
print("Training images:", x_train_full.shape)
print("Training labels:", y_train_full.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)

# Display sample images
plt.figure(figsize=(10, 4))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train_full[i], cmap="gray")
    plt.title(f"Label: {y_train_full[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
```




```
# Normalize pixel values to range 0 to 1
x_train_full = x_train_full.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# CNN expects image shape: height, width, channels
x_train_full = x_train_full[..., np.newaxis]
x_test = x_test[..., np.newaxis]

# Split training data into training and validation sets
x_train, x_val, y_train, y_val = train_test_split(
    x_train_full,
    y_train_full,
    test_size=5000,
    random_state=42,
    stratify=y_train_full
)

print("Training set:", x_train.shape)
print("Validation set:", x_val.shape)
print("Testing set:", x_test.shape)
```

```
Training set: (55000, 28, 28, 1)
Validation set: (5000, 28, 28, 1)
Testing set: (10000, 28, 28, 1)
```

```
# Build CNN model
model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),

    layers.Conv2D(
        32, kernel_size=(3, 3),
        activation="relu"
    ),
    layers.MaxPooling2D(pool_size=(2, 2)),

    layers.Conv2D(
        64, kernel_size=(3, 3),
        activation="relu"
    ),
```

```
    layers.MaxPooling2D(pool_size=(2, 2)),

    layers.Flatten(),

    layers.Dense(64, activation="relu"),
    layers.Dropout(0.3),

    layers.Dense(10, activation="softmax")
])

# Configure model
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Display architecture
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 64)	102,464
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)
Trainable params: 121,930 (476.29 KB)

```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=128,
    validation_data=(x_val, y_val),
    verbose=1
)

print("Model training completed!")
```

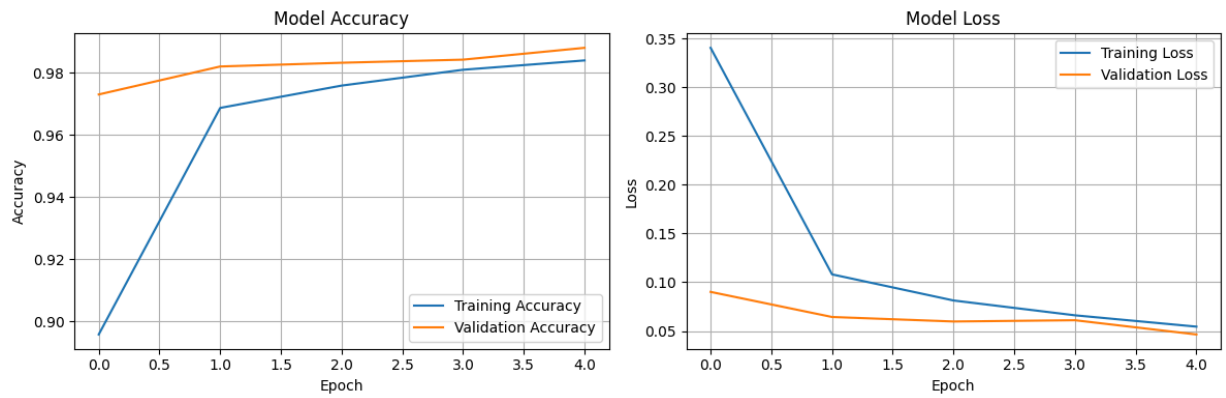
```
Epoch 1/5
430/430 ━━━━━━━━━━━━━━━━━━━━ 50s 112ms/step - accuracy: 0.8958 - loss: 0.3403 - val_accuracy:
Epoch 2/5
430/430 ━━━━━━━━━━━━━━━━━━━━ 46s 106ms/step - accuracy: 0.9687 - loss: 0.1080 - val_accuracy:
Epoch 3/5
430/430 ━━━━━━━━━━━━━━━━━━━━ 47s 110ms/step - accuracy: 0.9759 - loss: 0.0813 - val_accuracy:
Epoch 4/5
430/430 ━━━━━━━━━━━━━━━━━━━━ 47s 109ms/step - accuracy: 0.9809 - loss: 0.0661 - val_accuracy:
Epoch 5/5
430/430 ━━━━━━━━━━━━━━━━━━━━ 47s 109ms/step - accuracy: 0.9839 - loss: 0.0545 - val_accuracy:
Model training completed!
```

```
# Plot accuracy
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

# Plot loss
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```




```
# Evaluate model on test data
test_loss, test_accuracy = model.evaluate(
    x_test, y_test, verbose=0
)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

# Predict class probabilities and labels
y_prob = model.predict(x_test, verbose=0)
y_pred = np.argmax(y_prob, axis=1)

# Calculate evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(
    y_test, y_pred, average="macro", zero_division=0
)
recall = recall_score(
    y_test, y_pred, average="macro", zero_division=0
)
f1 = f1_score(
    y_test, y_pred, average="macro", zero_division=0
)
```

```
print("\nEvaluation Metrics")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-Score : {f1:.4f}")

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

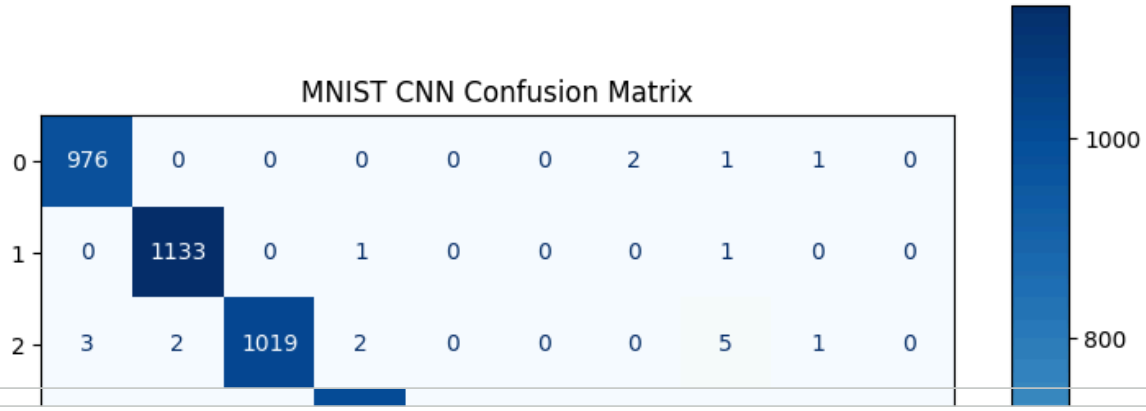
display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=np.arange(10)
)

fig, ax = plt.subplots(figsize=(9, 9))
display.plot(ax=ax, cmap="Blues", values_format="d")
plt.title("MNIST CNN Confusion Matrix")
plt.show()

# Detailed class-wise report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

```
Test Loss: 0.0327
Test Accuracy: 98.94%

Evaluation Metrics
Accuracy : 0.9894
Precision: 0.9894
Recall   : 0.9893
F1-Score : 0.9893
```




```
# Display predictions for 10 test images
plt.figure(figsize=(12, 5))

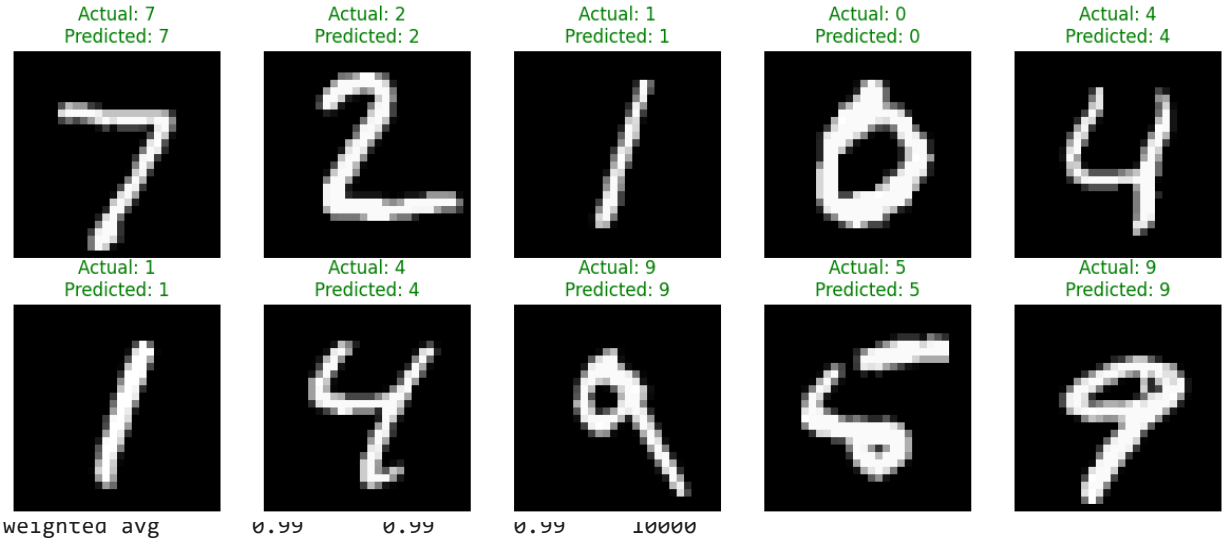
for i in range(10):
    predicted_digit = y_pred[i]
    actual_digit = y_test[i]

    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].squeeze(), cmap="gray")

    color = "green" if predicted_digit == actual_digit else "red"

    plt.title(
        f"Actual: {actual_digit}\nPredicted: {predicted_digit}",
        color=color
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
weighted avg       0.99      0.99      0.99     10000
```